

Архитектура приложений Flutter

Правильная архитектура обеспечивает масштабируемость, тестируемость и поддерживаемость приложения. Рассмотрим популярные архитектурные паттерны для Flutter.

Clean Architecture

Clean Architecture разделяет код на слои с чёткими зависимостями.

Зачем она нужна

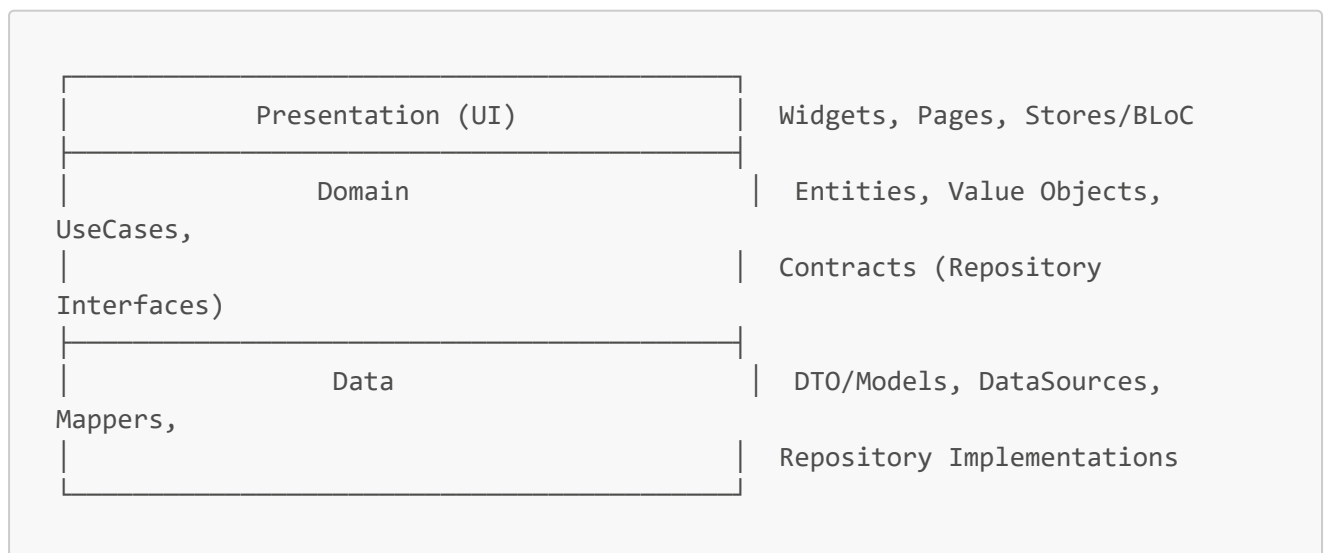
Clean Architecture обычно выбирают для средних и больших приложений, где важно, чтобы код:

- было проще тестировать (особенно бизнес-логику без Flutter и без сети)
- можно было заменять внешние детали (API-клиент, БД, кэш, навигацию) без переписывания ядра
- оставался понятным при росте команды и количества фич

Если упростить, то Clean Architecture — это способ управлять сложностью: мы заранее договариваемся, где «живут» правила предметной области, где «живёт» инфраструктура, и как они общаются.

Правило зависимостей (Dependency Rule)

Ключевая идея: зависимости направлены «внутрь», к бизнес-логике. То есть внешний мир (UI, HTTP, БД) может зависеть от домена, но домен не должен зависеть от внешнего мира.



Практический смысл: бизнес-логика (Domain) остаётся максимально «чистой» и независимой, поэтому её легче покрывать unit-тестами и переносить между проектами.

Как слои общаются друг с другом

Чтобы слои не «приклеивались» друг к другу, в Clean Architecture используют границы:

- **Use case boundary:** UI не вызывает Data напрямую. UI вызывает UseCase, который описывает сценарий.
- **Repository boundary:** Domain описывает интерфейс репозитория, а Data предоставляет реализацию.
- **Mapping boundary:** форматы данных на границах отличаются: API/БД → DTO/Model → Entity → UI-модель.

Отсюда рождается правило: объекты «для внешнего мира» (DTO, JSON, таблицы) не должны протекать в Domain и Presentation.

Типичные ошибки при внедрении

- Перетаскивать `UserDto` (или `UserModel`) в UI и делать `Text(userDto.email)` напрямую.
- Зависеть в `domain/` от `package:flutter/*`, `http`, `dio`, `shared_preferences` и т.п.
- Делать «use case» как тонкую прокладку, а всю логику держать в Store/BLoC или в Repository Impl.
- Раздувать `core/` до «помойки»: лучше выносить в `core/` только то, что реально переиспользуется между фичами.

Структура проекта

```
lib/
├── core/
│   ├── error/
│   │   ├── exceptions.dart
│   │   └── failures.dart
│   ├── usecases/
│   │   └── usecase.dart
│   └── utils/
│       └── constants.dart
├── features/
│   └── user/
│       ├── data/
│       │   ├── datasources/
│       │   │   ├── user_local_datasource.dart
│       │   │   └── user_remote_datasource.dart
│       │   ├── models/
│       │   │   └── user_model.dart
│       │   └── repositories/
│       │       └── user_repository_impl.dart
│       ├── domain/
│       │   ├── entities/
│       │   │   └── user.dart
│       │   ├── repositories/
│       │   │   └── user_repository.dart
│       │   └── usecases/
│       │       ├── get_user.dart
│       │       └── update_user.dart
│       └── presentation/
│           └── bloc/
```

```

├── user_bloc.dart
├── user_event.dart
├── user_state.dart
├── pages/
│   └── user_page.dart
├── widgets/
│   └── user_card.dart
└── injection_container.dart

```

Пример: (DDD + Clean Architecture)

В проекте Clean Architecture используется в стиле feature-first (DDD): код группируется по фичам (модулям), и внутри каждой фичи есть слои **data/domain/presentation**. Всё, что относится ко всему приложению, находится в **core/**.

Коротко по ответственности слоёв:

- **domain/** — бизнес-правила: сущности, контракты репозитория, use cases. Не зависит от Flutter, HTTP и БД.
- **data/** — работа с данными: DTO/модели, data sources (API/кэш), реализации репозитория, маппинг DTO ↔ Entity. Зависит от **domain/**.
- **presentation/** — UI и управление состоянием: страницы/виджеты, а также stores (используется MobX). Вызывает use cases и не знает деталей API.

```

lib/
├── core/
│   ├── injection/           # DI (get_it + injectable)
│   ├── routes/             # маршрутизация (GoRouter)
│   ├── services/          # сервисы (auth state, FCM, location)
│   ├── theme/             # тема/стили
│   ├── ui/               # дизайн-система/кит
│   └── utils/             # утилиты и обработка ошибок
├── features/
│   └── auth/
│       ├── data/          # Data Layer (DTO, data sources, impl repo,
mappers)
│       ├── domain/        # Domain Layer (entities, contracts, use
cases)
│       └── presentation/  # Presentation Layer (pages/widgets/stores)
└── main.dart

```

Главная идея Clean Architecture — зависимости направлены «внутрь»: **presentation** → **domain** ← **data**. Снаружи можно менять UI, сеть, хранилище, не переписывая бизнес-логику.

Поток данных в этой схеме обычно выглядит так:

```
UI → Store (MobX) → UseCase → Repository (Domain) → Repo Impl (Data) →
DataSource → API/DB
```

Entity vs DTO/Model vs UI-модель

Одна из самых полезных «теоретических» дисциплин в Clean Architecture — не смешивать модели разных уровней:

- **Entity (Domain)** — объект предметной области. Он существует потому, что он нужен бизнесу (пользователь, заказ, подписка), а не потому что так устроен API.
- **DTO/Model (Data)** — форма данных для внешних источников: JSON, ответ API, таблица в БД, запись в кэше. DTO удобно сериализовать/десериализовать, но в нём не должно быть бизнес-инвариантов.
- **UI-модель (Presentation)** — форма данных для конкретного экрана: объединённые поля, форматированные строки, флаги для отображения. Её можно строить из Entity, но UI-модель не должна утекать в Domain.

Почему это важно: как только UI начинает жить на DTO, любая правка контракта API начинает ломать экран и бизнес-логику одновременно.

Где хранить общие вещи (core)

core/ обычно содержит общие для всех фич части:

- базовые типы ошибок (например, **Failure**) и правила их отображения
- DI-конфигурацию (composition root), роутинг, тему, общий UI-kit
- инфраструктурные сервисы, которые не являются фичей сами по себе (например, FCM, геолокация)

Важно держать **core/** небольшим: если модуль используется только одной фичей — чаще логичнее оставить его внутри фичи.

Как проектировать Use Cases

Use case — это «сценарий использования» с точки зрения приложения: например, “войти по email”, “загрузить профиль”, “обновить настройки”. Хороший use case:

- принимает простые входные параметры (часто через **Params**-класс)
- возвращает результат в доменных терминах (Entity/Value Object) или доменную ошибку
- не знает, откуда пришли данные (сеть/кэш/БД) — это скрывает репозиторий

Domain Layer

Domain — это «сердце» приложения: здесь описываются правила предметной области и сценарии, которые не должны зависеть от UI и инфраструктуры. В Flutter-проектах это важно особенно сильно: UI быстро меняется, а бизнес-логика должна оставаться стабильной.

Что обычно находится в Domain:

- **Entities** — объекты с идентичностью (например, `User` с `id`), описывают бизнес-смысл.
- **Value Objects** — неизменяемые значения с валидацией (например, `Email`, `PhoneNumber`). Они помогают «защитить» инварианты в типы.
- **Use Cases** — сценарии использования (применение бизнес-правил). Это хороший центр для оркестрации: «получи данные», «проверь условие», «сохрани результат».
- **Repository interfaces** — контракты доступа к данным. Это «порт» (port): Domain описывает, что ему нужно, а Data предоставляет реализацию.

В примерах ниже используется подход с `Either<Failure, T>`: мы возвращаем не исключения наружу, а явный результат «успех/ошибка». Это делает ошибки частью контракта и упрощает тестирование и UI-обработку.

```
// Сущность (Entity)
class User {
    final String id;
    final String name;
    final String email;

    const User({
        required this.id,
        required this.name,
        required this.email,
    });
}

// Абстрактный репозиторий
abstract class UserRepository {
    Future<Either<Failure, User>> getUser(String id);
    Future<Either<Failure, void>> updateUser(User user);
    Future<Either<Failure, List<User>>> getAllUsers();
}

// UseCase
abstract class UseCase<Type, Params> {
    Future<Either<Failure, Type>> call(Params params);
}

class GetUser implements UseCase<User, GetUserParams> {
    final UserRepository repository;

    GetUser(this.repository);

    @override
    Future<Either<Failure, User>> call(GetUserParams params) {
        return repository.getUser(params.id);
    }
}

class GetUserParams {
    final String id;
}
```

```
    GetUserParams({required this.id});  
}
```

Data Layer

Data Layer отвечает за получение/сохранение данных и адаптацию внешних форматов под доменные сущности.

Типовые роли в Data:

- **DataSource** — конкретный источник данных (HTTP, БД, кэш). Важно, что DataSource не «знает» про UI.
- **DTO/Model** — объект для сериализации/десериализации (JSON ↔ объект). Его удобно хранить и передавать по сети.
- **Mapper** — преобразование DTO ↔ Entity. Это отдельная граница, которая защищает Domain от структуры API.
- **Repository implementation** — сборка стратегии: откуда взять данные (сеть/кэш), как обработать ошибки и когда синхронизировать.

В примере `UserModel extends User` — это упрощение. В реальных проектах часто полезнее держать DTO отдельно от Entity, чтобы изменения в API не «протекали» в доменную модель и не ломали бизнес-правила.

```
// Модель данных  
class UserModel extends User {  
    const UserModel({  
        required String id,  
        required String name,  
        required String email,  
    }) : super(id: id, name: name, email: email);  
  
    factory UserModel.fromJson(Map<String, dynamic> json) {  
        return UserModel(  
            id: json['id'],  
            name: json['name'],  
            email: json['email'],  
        );  
    }  
  
    Map<String, dynamic> toJson() {  
        return {  
            'id': id,  
            'name': name,  
            'email': email,  
        };  
    }  
}
```

```
// Remote Data Source
abstract class UserRemoteDataSource {
    Future<UserModel> getUser(String id);
    Future<void> updateUser(UserModel user);
}

class UserRemoteDataSourceImpl implements UserRemoteDataSource {
    final http.Client client;

    UserRemoteDataSourceImpl({required this.client});

    @override
    Future<UserModel> getUser(String id) async {
        final response = await client.get(
            Uri.parse('$baseUrl/users/$id'),
        );

        if (response.statusCode == 200) {
            return UserModel.fromJson(json.decode(response.body));
        } else {
            throw ServerException();
        }
    }

    @override
    Future<void> updateUser(UserModel user) async {
        final response = await client.put(
            Uri.parse('$baseUrl/users/${user.id}'),
            body: json.encode(user.toJson()),
        );

        if (response.statusCode != 200) {
            throw ServerException();
        }
    }
}

// Repository Implementation
class UserRepositoryImpl implements UserRepository {
    final UserRemoteDataSource remoteDataSource;
    final UserLocalDataSource localDataSource;
    final NetworkInfo networkInfo;

    UserRepositoryImpl({
        required this.remoteDataSource,
        required this.localDataSource,
        required this.networkInfo,
    });

    @override
    Future<Either<Failure, User>> getUser(String id) async {
        if (await networkInfo.isConnected) {
            try {

```

```

        final user = await remoteDataSource.getUser(id);
        await localDataSource.cacheUser(user);
        return Right(user);
    } on ServerException {
        return Left(ServerFailure());
    }
  } else {
    try {
      final user = await localDataSource.getCachedUser(id);
      return Right(user);
    } on CacheException {
      return Left(CacheFailure());
    }
  }
}
}

```

Presentation Layer (BLoC)

Presentation Layer — это слой экрана: виджеты + управление состоянием. Его задача — превратить доменные сценарии в понятные UI-состояния (загрузка/данные/ошибка) и вызывать use cases.

Если вы используете BLoC, удобно мыслить как конечным автоматом:

- **Event** — действие пользователя или сигнал системы (нажали кнопку, открыли экран).
- **State** — то, что нужно UI для отрисовки (включая флаги загрузки и данные).

Важно: состояния должны быть ориентированы на UI, а не повторять DTO из API.

```

// События
abstract class UserEvent {}

class GetUserEvent extends UserEvent {
  final String id;
  GetUserEvent(this.id);
}

class UpdateUserEvent extends UserEvent {
  final User user;
  UpdateUserEvent(this.user);
}

// Состояния
abstract class UserState {}

class UserInitial extends UserState {}

class UserLoading extends UserState {}

class UserLoaded extends UserState {

```



```

    final User user;
    UserLoaded(this.user);
}

class UserError extends UserState {
    final String message;
    UserError(this.message);
}

// BLoC
class UserBloc extends Bloc<UserEvent, UserState> {
    final GetUser getUser;
    final UpdateUser updateUser;

    UserBloc({
        required this.getUser,
        required this.updateUser,
    }) : super(UserInitial()) {
        on<GetUserEvent>(_onGetUser);
        on<UpdateUserEvent>(_onUpdateUser);
    }

    Future<void> _onGetUser(
        GetUserEvent event,
        Emitter<UserState> emit,
    ) async {
        emit(UserLoading());

        final result = await getUser(GetUserParams(id: event.id));

        result.fold(
            (failure) => emit(UserError(_mapFailureToMessage(failure))),
            (user) => emit(UserLoaded(user)),
        );
    }

    String _mapFailureToMessage(Failure failure) {
        switch (failure.runtimeType) {
            case ServerFailure:
                return 'Ошибка сервера';
            case CacheFailure:
                return 'Ошибка кэша';
            default:
                return 'Неизвестная ошибка';
        }
    }
}

```

MVVM (Model-View-ViewModel)

MVVM часто применяют, когда хочется более «простого» отделения UI от логики без строгого разделения слоёв по Clean Architecture. При этом ViewModel выступает прослойкой между UI и сервисами/репозиториями и хранит состояние, удобное для отображения.

Роли в MVVM:

- **View** — виджеты Flutter.
- **ViewModel** — состояние экрана и команды (методы) для UI.
- **Model** — данные (не обязательно доменные entities; часто просто модели приложения).
- **Service** — взаимодействие с сетью/хранилищем.

Структура

```
lib/  
├── models/  
│   └── user.dart  
├── services/  
│   └── user_service.dart  
├── viewmodels/  
│   └── user_viewmodel.dart  
└── views/  
    └── user_view.dart
```

Реализация

```
// Model  
class User {  
  final String id;  
  final String name;  
  final String email;  
  
  User({required this.id, required this.name, required this.email});  
  
  factory User.fromJson(Map<String, dynamic> json) {  
    return User(  
      id: json['id'],  
      name: json['name'],  
      email: json['email'],  
    );  
  }  
}  
  
// Service  
class UserService {  
  final http.Client _client;  
  
  UserService(this._client);
```

```

Future<User> getUser(String id) async {
    final response = await _client.get(Uri.parse('$baseUrl/users/$id'));
    return User.fromJson(json.decode(response.body));
}

Future<List<User>> getUsers() async {
    final response = await _client.get(Uri.parse('$baseUrl/users'));
    final List data = json.decode(response.body);
    return data.map((json) => User.fromJson(json)).toList();
}
}

// ViewModel
class UserViewModel extends ChangeNotifier {
    final UserService _userService;

    User? _user;
    List<User> _users = [];
    bool _isLoading = false;
    String? _error;

    User? get user => _user;
    List<User> get users => _users;
    bool get isLoading => _isLoading;
    String? get error => _error;

    UserViewModel(this._userService);

    Future<void> loadUser(String id) async {
        _isLoading = true;
        _error = null;
        notifyListeners();

        try {
            _user = await _userService.getUser(id);
        } catch (e) {
            _error = e.toString();
        } finally {
            _isLoading = false;
            notifyListeners();
        }
    }

    Future<void> loadUsers() async {
        _isLoading = true;
        _error = null;
        notifyListeners();

        try {
            _users = await _userService.getUsers();
        } catch (e) {
            _error = e.toString();
        } finally {

```

```

        _isLoading = false;
        notifyListeners();
    }
}

// View
class UserView extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return ChangeNotifierProvider(
      create: (_) => UserViewModel(context.read<UserService>())..loadUsers(),
      child: Consumer<UserViewModel>(
        builder: (context, viewModel, child) {
          if (viewModel.isLoading) {
            return Center(child: CircularProgressIndicator());
          }

          if (viewModel.error != null) {
            return Center(child: Text('Ошибка: ${viewModel.error}'));
          }

          return ListView.builder(
            itemCount: viewModel.users.length,
            itemBuilder: (context, index) {
              final user = viewModel.users[index];
              return ListTile(
                title: Text(user.name),
                subtitle: Text(user.email),
              );
            },
          );
        },
      ),
    );
  }
}

```

Repository Pattern

Repository Pattern — это способ скрыть детали хранения данных за интерфейсом. Для Clean Architecture репозиторий важен как граница: Domain зависит от интерфейса репозитория, а Data предоставляет реализацию.

Полезные критерии:

- Репозиторий отвечает за **доступ к данным в терминах домена**, а не за детали транспорта.
- DataSource отвечает за **конкретный источник** (HTTP/SQLite/Preferences).
- Маппинг DTO ↔ Entity лучше держать рядом с Data или отдельно, но не в UI.

```

// Абстрактный репозиторий
abstract class Repository<T> {
    Future<List<T>> getAll();
    Future<T?> getById(String id);
    Future<void> create(T item);
    Future<void> update(T item);
    Future<void> delete(String id);
}

// Реализация
class UserRepository implements Repository<User> {
    final ApiClient _apiClient;
    final LocalStorage _localStorage;

    UserRepository(this._apiClient, this._localStorage);

    @override
    Future<List<User>> getAll() async {
        try {
            final users = await _apiClient.getUsers();
            await _localStorage.saveUsers(users);
            return users;
        } catch (e) {
            // Вернуть кэшированные данные при ошибке
            return _localStorage.getUsers();
        }
    }

    @override
    Future<User?> getById(String id) async {
        try {
            return await _apiClient.getUser(id);
        } catch (e) {
            return _localStorage.getUser(id);
        }
    }

    @override
    Future<void> create(User user) async {
        await _apiClient.createUser(user);
        await _localStorage.saveUser(user);
    }

    @override
    Future<void> update(User user) async {
        await _apiClient.updateUser(user);
        await _localStorage.saveUser(user);
    }

    @override
    Future<void> delete(String id) async {
        await _apiClient.deleteUser(id);
    }
}

```

```
    await _localStorage.deleteUser(id);  
  }  
}
```

Dependency Injection

Dependency Injection (DI) нужен, чтобы «склеить» приложение без жёстких зависимостей на конкретные реализации. В Clean Architecture точка сборки называется composition root: именно там выбирается, какие реализации использовать, и как они создаются.

В Flutter DI часто организуют так:

- **registerFactory** — новый экземпляр при каждом запросе (подходит для BLoC/Store).
- **registerLazySingleton** — один экземпляр на всё приложение (подходит для репозитория, клиентов, сервисов).

get_it

```
import 'package:get_it/get_it.dart';  
  
final sl = GetIt.instance;  
  
Future<void> init() async {  
  // BLoCs  
  sl.registerFactory(  
    () => UserBloc(getUser: sl(), updateUser: sl()),  
  );  
  
  // Use cases  
  sl.registerLazySingleton(() => GetUser(sl()));  
  sl.registerLazySingleton(() => UpdateUser(sl()));  
  
  // Repository  
  sl.registerLazySingleton<UserRepository>(  
    () => UserRepositoryImpl(  
      remoteDataSource: sl(),  
      localDataSource: sl(),  
      networkInfo: sl(),  
    ),  
  );  
  
  // Data sources  
  sl.registerLazySingleton<UserRemoteDataSource>(  
    () => UserRemoteDataSourceImpl(client: sl()),  
  );  
  sl.registerLazySingleton<UserLocalDataSource>(  
    () => UserLocalDataSourceImpl(sharedPreferences: sl()),  
  );  
  
  // External
```

```

    final sharedPreferences = await SharedPreferences.getInstance();
    sl.registerLazySingleton(() => sharedPreferences);
    sl.registerLazySingleton(() => http.Client());
    sl.registerLazySingleton<NetworkInfo>(() => NetworkInfoImpl(sl()));
  }

  // Использование
  void main() async {
    WidgetsFlutterBinding.ensureInitialized();
    await init();
    runApp(MyApp());
  }

  class MyApp extends StatelessWidget {
    @override
    Widget build(BuildContext context) {
      return MaterialApp(
        home: BlocProvider(
          create: (_) => sl<UserBloc>(),
          child: UserPage(),
        ),
      );
    }
  }
}

```

injectable

```

import 'package:injectable/injectable.dart';
import 'package:get_it/get_it.dart';

final getIt = GetIt.instance;

@InjectableInit()
void configureDependencies() => getIt.init();

// Анотації
@injectable
class UserService {
  final ApiClient _client;

  UserService(this._client);
}

@lazySingleton
class ApiClient {
  final http.Client _httpClient;

  ApiClient(this._httpClient);
}

```

```

@module
abstract class RegisterModule {
    @lazySingleton
    http.Client get httpClient => http.Client();

    @preResolve
    Future<SharedPreferences> get prefs => SharedPreferences.getInstance();
}

```

State Management Patterns

Выбор управления состоянием влияет на то, где живёт логика экрана и как UI реагирует на изменения. Какой бы подход вы ни выбрали (BLoC, MobX, Provider, Cubit и т.п.), полезно держать инвариант: UI обращается к Domain через use cases и получает состояния, подходящие для отображения.

BLoC Pattern

```

// События
sealed class CounterEvent {}

class IncrementEvent extends CounterEvent {}
class DecrementEvent extends CounterEvent {}
class ResetEvent extends CounterEvent {}

// Состояние
class CounterState {
    final int count;
    final bool isLoading;

    const CounterState({
        this.count = 0,
        this.isLoading = false,
    });

    CounterState copyWith({int? count, bool? isLoading}) {
        return CounterState(
            count: count ?? this.count,
            isLoading: isLoading ?? this.isLoading,
        );
    }
}

// BLoC
class CounterBloc extends Bloc<CounterEvent, CounterState> {
    CounterBloc() : super(const CounterState()) {
        on<IncrementEvent>((event, emit) {
            emit(state.copyWith(count: state.count + 1));
        });
    }
}

```



```

    on<DecrementEvent>((event, emit) {
      emit(state.copyWith(count: state.count - 1));
    });

    on<ResetEvent>((event, emit) {
      emit(const CounterState());
    });
  }
}

```

Cubit Pattern

```

class CounterCubit extends Cubit<int> {
  CounterCubit() : super(0);

  void increment() => emit(state + 1);
  void decrement() => emit(state - 1);
  void reset() => emit(0);
}

// Использование
BlocBuilder<CounterCubit, int>(<
  builder: (context, count) {
    return Text('$count');
  },
)

```

Error Handling

В больших приложениях выгодно различать «технические исключения» и «доменные ошибки», которые понимает UI:

- **Exception** — деталь реализации (ошибка сети, парсинга, БД). Обычно возникает в DataSource.
- **Failure/Result** — то, что можно безопасно показать UI (и протестировать). Обычно формируется на уровне Repository/UseCase.

Так UI не обязан знать про `DioError` или `SocketException`: он работает с `NetworkFailure`, `ServerFailure` и т.п.

```

// Базовый класс ошибки
abstract class Failure {
  final String message;
  const Failure(this.message);
}

class ServerFailure extends Failure {
  const ServerFailure([String message = 'Ошибка сервера']) : super(message);
}

```

```

}

class CacheFailure extends Failure {
  const CacheFailure([String message = 'Ошибка кэша']) : super(message);
}

class NetworkFailure extends Failure {
  const NetworkFailure([String message = 'Нет соединения']) : super(message);
}

// Either для обработки результатов
import 'package:dartz/dartz.dart';

Future<Either<Failure, User>> getUser(String id) async {
  try {
    final user = await remoteDataSource.getUser(id);
    return Right(user);
  } on ServerException {
    return Left(ServerFailure());
  } on NetworkException {
    return Left(NetworkFailure());
  }
}

// Обработка в UI
result.fold(
  (failure) => showError(failure.message),
  (user) => showUser(user),
);

```

Testing Architecture

Преимущество слоистой архитектуры — тесты становятся проще и быстрее:

- **Domain:** unit-тесты на use cases и value objects без Flutter и без моков сети.
- **Data:** тесты на мапперы, репозитории и data sources (часто с моками клиента/хранилища).
- **Presentation:** тесты на состояния (BLoC/Store) и widget-тесты на отрисовку при разных состояниях.

```

// Mock repository
class MockUserRepository extends Mock implements UserRepository {}

// Unit-тест для UseCase
void main() {
  late GetUser usecase;
  late MockUserRepository mockRepository;

  setUp(() {
    mockRepository = MockUserRepository();
    usecase = GetUser(mockRepository);
  });
}

```

```

});

final tUser = User(id: '1', name: 'Test', email: 'test@test.com');

test('должен получить пользователя из репозитория', () async {
  // arrange
  when(() => mockRepository.getUser(any()))
    .thenAnswer((_) async => Right(tUser));

  // act
  final result = await usecase(GetUserParams(id: '1'));

  // assert
  expect(result, Right(tUser));
  verify(() => mockRepository.getUser('1'));
});
}

// Widget-тест с Bloc
void main() {
  late UserBloc bloc;

  setUp(() {
    bloc = MockUserBloc();
  });

  testWidgets('показывает пользователя после загрузки', (tester) async {
    when(() => bloc.state).thenReturn(UserLoaded(tUser));

    await tester.pumpWidget(
      BlocProvider.value(
        value: bloc,
        child: MaterialApp(home: UserPage()),
      ),
    );

    expect(find.text('Test'), findsOneWidget);
  });
}

```

Лучшие практики

1. **Разделяйте слои** — Domain не должен зависеть от Data или Presentation.
2. **Используйте Dependency Injection** — для тестируемости и гибкости.
3. **Обрабатывайте ошибки явно** — используйте Either или sealed classes.
4. **Пишите тесты** — отдельно для каждого слоя.
5. **Документируйте архитектурные решения** — для команды.

Вывод

Правильная архитектура — это инвестиция в будущее проекта. Clean Architecture, MVVM и другие паттерны помогают создавать масштабируемые и поддерживаемые приложения. Выбор архитектуры зависит от размера проекта и требований команды.